

Jasmin Alizadeh-Shabdiz

CDSDS 210

Final Project

A. Project Overview

Goal:

The project answers the question: Which countries are structurally central in a global network based on consumer prices? It identifies and compares the centrality of countries using degree, closeness, and betweenness metrics in a graph where nodes are countries and edges represent similar price values.

Dataset:

- **Source:** <https://data.un.org/Data.aspx?q=food&d=LABORSTA&f=tableCode%3a7C>
 - **File name:** prices.csv
 - **Size:** ~1–2 MB
 - **Format:** TSV (Tab-Separated Values)
 - **Note:** Due to size, the file is not hosted on GitHub but should be placed in the project directory.
-

B. Data Processing

Loading:

- Used the csv crate with `ReaderBuilder::new().delimiter(b'\t')` to read the TSV file.
- Employed `serde` with the `Deserialize` trait to parse rows into a custom struct.

Cleaning and Transformations:

- Removed commas in number strings using `.replace(",", "")`.

- Parsed cleaned strings into f64 using `.parse()`.
 - Grouped data by country, computing average price values.
 - Built a graph by connecting countries whose average values differ by less than 100.0.
-

C. Code Structure

Modules

The project uses a single file (`main.rs`) but logically separates responsibilities into sections:

1. **Data Cleaning & Loading** – Reads and parses the input dataset.
2. **Graph Construction** – Builds an adjacency matrix based on price similarity.
3. **Centrality Metrics** – Calculates degree, closeness, and betweenness.
4. **User Interaction** – Accepts country input and displays metrics.

Key Functions & Types

EmploymentRecord (Struct)

- **Purpose:** Represents a row of the dataset.
- **Fields:** `country: String`, `value: Option<String>`
- **Usage:** Used with `serde` to deserialize CSV rows.

`clean_value(value: &Option<String>) -> Option<f64>`

- **Purpose:** Parses numeric strings to floats after removing formatting.
- **Inputs:** Reference to an optional string value.
- **Outputs:** Optional floating-point number.
- **Logic:** Strips commas and parses as `f64`.

`build_adjacency_matrix(...) -> Vec<Vec<i32>>`

- **Purpose:** Constructs a graph of similar countries.
- **Inputs:**
 - countries: List of country names
 - averages: Map from country to average value
 - threshold: Max difference to connect two countries
- **Output:** Adjacency matrix representing the graph.
- **Logic:** Connects countries with similar average values.

degree centrality(matrix: &Vec<Vec<i32>>) -> Vec<usize>

- **Purpose:** Computes number of connections for each country.
- **Logic:** Sums entries per row in the adjacency matrix.

closeness centrality(matrix: &Vec<Vec<i32>>) -> Vec<f64>

- **Purpose:** Measures how close a node is to others on average.
- **Logic:** Runs BFS from each node, averages path lengths.

betweenness centrality(matrix: &Vec<Vec<i32>>) -> Vec<f64>

- **Purpose:** Calculates how often a node lies on shortest paths.
- **Logic:** For every node pair, find shortest paths and count how often each node is used.

Main Workflow

1. Parse and clean the dataset into a hashmap of country -> average value.
2. Build a similarity graph by connecting close countries.
3. Compute centrality scores using graph algorithms.
4. Prompt users for a country and print its scores compared to global averages.

D. Tests

Command

cargo test

Output

```
running 7 tests
test centrality::tests::test_betweenness centrality ... ok
test centrality::tests::test_closeness centrality ... ok
test data::tests::test_clean_value ... ok
test centrality::tests::test_degree centrality ... ok
test graph::tests::test_build_adjacency_matrix_all_connected ... ok
test graph::tests::test_build_adjacency_matrix_basic ... ok
test data::tests::test_read_employment_data_manual_file ... ok

test result: ok. 7 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s
```

Test Details

Module: centrality.rs

- **Test_degree centrality**
 - Checks that direct connections (node degrees) are computed correctly.
 - Validates core graph building logic.
- **Test_closeness centrality**
 - Uses a triangle graph to confirm equal closeness values.
 - Confirms BFS and average path-length logic.
- **Test_betweenness centrality**
 - Uses a path graph to verify that middle nodes have higher betweenness.
 - Ensures shortest path enumeration and contribution counting works as expected.

Module: data.rs

- **Test_clean_value**
 - Verifies proper parsing of numeric strings with commas and invalid inputs.
 - Ensures robustness in data cleaning for downstream calculations.
 - **Test_read_employment_data_manual_file**
 - Creates a temporary file to confirm averages are computed per country.
 - Validates correct CSV parsing, grouping, and averaging logic.
-

Module: graph.rs

- **Test_build_adjacency_matrix_basic**
 - Tests correct edge creation based on threshold differences between country averages.
 - Checks edge presence/absence and self-connection exclusion.
 - **Test_build_adjacency_matrix_all_connected**
 - Ensures all countries are connected when the threshold is very high.
 - Validates general matrix construction and symmetry of connections.
-

Module: centrality.rs

- **test_degree_centrality**
 - Checks that direct connections (node degrees) are computed correctly.
 - Validates core graph building logic.
- **test_closeness_centrality**
 - Uses a triangle graph to confirm equal closeness values.
 - Confirms BFS and average path-length logic.
- **test_betweenness_centrality**

- Uses a path graph to verify that middle nodes have higher betweenness.
- Ensures shortest path enumeration and contribution counting works as expected.

E. Results

Sample Output:

```
Enter a country: Germany

--- Selected Country Centrality Report ---
Country: Germany
Degree Centrality: 207 (Global Avg: 197.69)
Closeness Centrality: 1.0190 (Global Avg: 2.1402)
Betweenness Centrality: 1.9665 (Global Avg: 2.7963)
```

Interpretation:

Germany appears more central than average in all metrics. This suggests its price data is similar to many others (high degree), quickly reachable (high closeness), and plays a key role in connecting others (high betweenness).

F. Usage Instructions

Build

cargo build

Run

cargo run

Interaction

The program will prompt:

“Enter a country:”

Type the full country name (e.g., Mexico) and press Enter.

Notes

- Case-insensitive matching is supported.
- Runtime: ~1–2 seconds for typical datasets (~200 countries).
- No command-line arguments required.

G. AI-Assistance Disclosure and Other Citations

AI Assistance

- My file was a TSV file not a CSV file, and I was not sure the difference between loading and reading a CSV file versus a TSV file. I asked chat gpt if there was a difference and how to specifically load a tsv file and chat gpt gave me two different ways to do it. But both versions had the `.delimiter(b't')` line that specifies that it is a TSV file.

What do I do when loading my csv file and its actually tsv and not csv

If you accidentally treat a TSV (Tab-Separated Values) file as a CSV (Comma-Separated Values) file, you just need to adjust the delimiter when reading the file. In Rust, if you're using a library like `csv` or `serde`, you can specify the delimiter explicitly. Here's how to handle it:

1. Using the `csv` crate: You can set the delimiter to a tab character (`\t`) when reading the file.

```
rust
use csv::ReaderBuilder;
use std::fs::File;

fn read_tsv(file_path: &str) -> Result<(), Box<dyn std::error::Error>> {
    let file = File::open(file_path)?;
    let mut rdr = ReaderBuilder::new()
        .delimiter(b't') // Specify tab delimiter
        .from_reader(file);

    for result in rdr.records() {
        let record = result?;
        println!("{}", record);
    }

    Ok(())
}

fn main() {
    if let Err(err) = read_tsv("data.tsv") {
        eprintln!("Error reading file: {err}");
    }
}
```

Using the `serde` crate: When using `serde` for deserialization, you can also specify the delimiter when reading the file.

```
rust
use serde::Deserialize;
use csv::ReaderBuilder;
use std::fs::File;

#[derive(Debug, Deserialize)]
struct Record {
    // Define the fields based on your TSV structure
    country: String,
    year: i32,
    region: String,
    value: f64,
}

fn read_tsv_with_serde(file_path: &str) -> Result<(), Box<dyn std::error::Error>> {
    let file = File::open(file_path)?;
    let mut rdr = ReaderBuilder::new()
        .delimiter(b't') // Specify tab delimiter
        .from_reader(file);

    for result in rdr.deserialize() {
        let record: Record = result?;
        println!("{}", record);
    }

    Ok(())
}

fn main() {
    if let Err(err) = read_tsv_with_serde("data.tsv") {
        eprintln!("Error reading file: {err}");
    }
}
```

- I also had a similar question for parsing a TSV file. I asked chat gpt how to parse a TSV file and chat gpt taught me 3 different ways to do it. The third way to do it was defining a

struct and deserializing which is the way I did it in my code. The way it works is serde automatically maps each row in the TSV file to a struct

how do I parse a tsv file

To parse a TSV (Tab-Separated Values) file in Rust, you can use the `csv` crate, which also supports TSV files by simply specifying the delimiter as a tab character. Here's a step-by-step guide to parsing a TSV file using the `csv` crate:

1. Add the `csv` crate to your `Cargo.toml`:

You'll need to add the `csv` crate to your project dependencies.

```
toml
[dependencies]
csv = "1.1"
```

2. Write code to parse a TSV file:

Now you can write a Rust function to read the TSV file and parse its contents. Below is a simple example of how to do this.

```
rust
use csv::ReaderBuilder;
use std::fs::File;
use std::error::Error;

fn read_tsv(file_path: &str) -> Result<(), Box<dyn Error>> {
    let file = File::open(file_path)?;

    // Set the delimiter to tab (b'\t')
    let mut rdr = ReaderBuilder::new()
        .delimiter(b'\t') // Specifies the tab delimiter
        .from_reader(file);
    ↓
```

3. Using Structs with Serde (Optional):

If your TSV file has a well-defined structure and you'd like to map it directly to a Rust struct, you can use the `serde` crate. Here's how:

1. Add the `serde` and `serde_derive` crates to `Cargo.toml` for deserialization:

```
toml
[dependencies]
csv = "1.1"
serde = { version = "1.0", features = ["derive"] }
serde_derive = "1.0"
```

2. Define a Struct and Deserialize:

```
rust
use serde::Deserialize;
use csv::ReaderBuilder;
use std::fs::File;
use std::error::Error;

#[derive(Debug, Deserialize)]
struct Record {
    country: String,
    year: i32,
    region: String,
    value: f64,
}

fn read_tsv_with_serde(file_path: &str) -> Result<(), Box<dyn Error>> {
    let file = File::open(file_path)?;

    // Set the delimiter to tab (b'\t')
    let mut rdr = ReaderBuilder::new()
    ↓
```